Práctica 1: CPU básica monociclo

Asignatura: Arquitectura e Ingeniería de Computadores para IA
Grado en Inteligencia Artificial

Resumen

En esta práctica, el alumno estudiará el funcionamiento de una CPU básica monociclo. Utilizando este modelo de microarquitectura, se ejecutará un programa en ensamblador RISC-V para observar la ruta de los datos y la ejecución de cada tipo básico de instrucción: `load`, `store`, `ALU` y `branch`, con el objetivo de entender su funcionamiento en detalle. Como herramienta, se hará uso del simulador de código abierto Ripes. Se responderán cuestiones sobre la ejecución del programa que ayudarán al alumno a alcanzar los objetivos de aprendizaje expuestos más abajo.

Objetivos de aprendizaje

Al finalizar esta sesión de laboratorio, el alumno habrá de haber alcanzado los siguientes objetivos de aprendizaje:

O1. Familiarizarse con el manejo del entorno de simulación: Adquirir soltura con la interfaz y la configuración del simulador Ripes. Aprender a configurar y utilizar el simulador para la ejecución y depuración de código ensamblador RISC-V.

O2. Entender los tipos básicos de instrucciones: Profundizar en el comportamiento de las cuatro categorías básicas de instrucciones (`load`, `store`, `ALU` y `branch`), observando su impacto directo en los registros, la memoria y la ruta de datos.

O3. Entender las señales internas de control: Ser capaz de entender las distintas señales internas de control de los procesadores, resultados intermedios y ejecución de las instrucciones.

Índice

1. Introducción a RISC-V y al simulador Ripes **3**
1.1. Puesta en marcha del simulador 3
1.2. Interfaz del simulador 4
1.2.1. Configuración del simulador 4
1.2.2. Carga del programa 4
1.2.3. Control de la simulación 5

2. Tipos de instrucciones y etapas de la ruta monociclo **6**
2.1. Tipos de instrucciones 6
2.2. Etapas de las instrucciones 7

3. Programa ejemplo **8**

4. Ejercicios **10**
4.1. Ejercicio 1: mapa de memoria inicial 10
4.2. Ejercicio 2: efecto de la ejecución de las instrucciones en registros y memoria 11
4.3. Ejercicio 3: funcionamiento interno del pipeline de la CPU 13

1. Introducción a RISC-V y al simulador Ripes

RISC-V [1] es un conjunto de instrucciones (ISA) de código abierto diseñado siguiendo la filosofía de los juegos de instrucciones reducidos (RISC). Fue propuesta originalmente por la Universidad de California, Berkeley. Se trata de una ISA modular, extensible y simple, lo que la convierte en un conjunto de instrucciones útil y eficaz para su implementación en un amplio abanico de sistemas, tanto a nivel académico como industrial. Para el desarrollo de esta práctica se explorará el subconjunto de instrucciones RV32I, enfocado en operaciones con enteros, accesos a memoria y control de flujo de ejecución en procesadores de 32 bits.

Para el desarrollo de las prácticas utilizaremos Ripes [2, 3]. Se trata de un simulador de CPU para RISC-V con interfaz gráfica diseñado para docencia. Mediante este simulador podemos visualizar en tiempo real, ciclo a ciclo, la ejecución detallada de las instrucciones en una CPU, exponiendo claramente la ruta de los datos a través de los distintos componentes del procesador. También permite evaluar distintos modelos de microarquitectura. En concreto, en esta práctica nos centraremos en un diseño de CPU básica monociclo. En este diseño, cada instrucción tarda siempre un ciclo de reloj en ejecutarse. Bajo este diseño, estudiaremos los tipos de instrucciones, la ruta de los datos y la ejecución de instrucciones.

1.1. Puesta en marcha del simulador

Ripes es un simulador de código abierto disponible en su repositorio público de Github[1]. Dicho simulador cuenta tanto con una versión web como con versiones estables descargables desde dicho repositorio.

- **Ejecución local de Ripes:** Para las prácticas se ha dejado en PoliformaT de la asignatura un archivo AppImage, que contiene el simulador en su versión v2.2.6.

  Una vez descargado el fichero, nos situamos en la carpeta que lo contiene y le otorgamos permisos de ejecución mediante el siguiente comando en la terminal:

  ```
  $ chmod a+x Ripes*.AppImage
  ```

  Una vez concedidos los permisos de ejecución, ejecutamos el simulador con el comando:

  ```
  $ ./Ripes*.AppImage
  ```

- **Ejecución web de Ripes:** Ripes cuenta con una versión web disponible a través del siguiente enlace:

  https://ripes.dk/

  Esta versión sólo se recomienda si no es posible acceder a la versión en PoliformaT, ya que podría diferir de la versión que se explica en la práctica.

[1] https://github.com/mortbopet/ripes

1.2. Interfaz del simulador

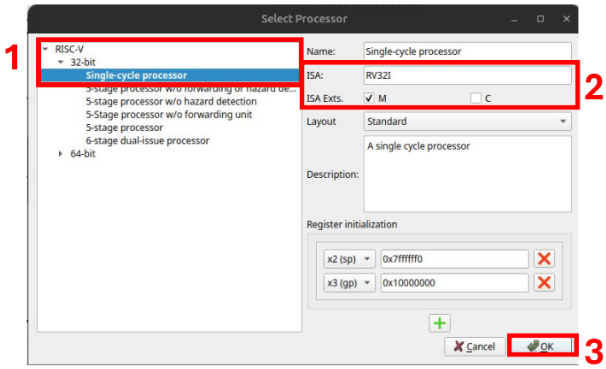
Una vez con el simulador abierto, se configurará para la simulación de una CPU RISC-V RV32I no segmentada monociclo. En esta sección se explica cómo configurarlo, cargar el programa ejemplo que usaremos en esta práctica y controlar la ejecución del programa.

1.2.1. Configuración del simulador

Configuramos el modelo de CPU como no segmentada. Para ello, en el menú superior seleccionaremos la opción de `Select processor` como se muestra en la siguiente imagen:

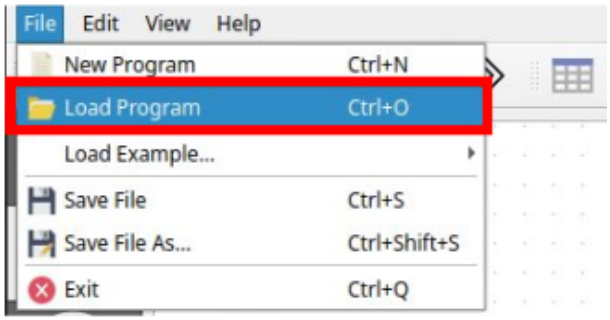


Se desplegará un menú con las opciones de procesadores disponibles. En dicho menú hemos de marcar en los modelos RISC-V de 32 bits la opción de: `Single-cycle processor`. En la parte de la derecha hemos de asegurarnos de que la ISA sea `RV32I`. Una vez configurado como se muestra en la imagen inferior, aceptamos los cambios.

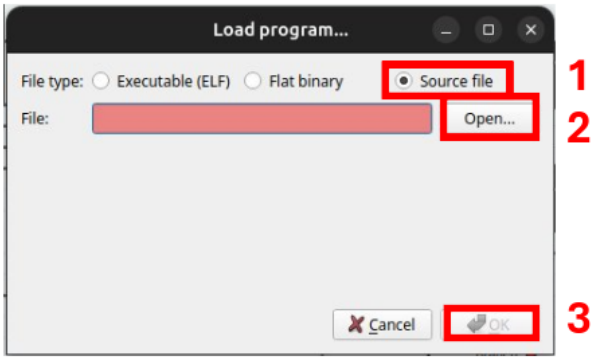


1.2.2. Carga del programa

El siguiente paso es cargar el programa `Prac01.s` en el simulador. Dicho fichero se encuentra en PoliformaT. Para ello, en el menú superior, dentro del desplegable `File`, seleccionamos la opción `Load Program`.



En el menú que se abre a continuación, nos aseguramos de seleccionar la opción de `Source File`, una vez seleccionada, le damos al botón `Open` y buscamos el programa a ejecutar. Una vez seleccionado, aceptamos.



Con el programa cargado, para visualizarlo, en el menú de la izquierda iremos a la vista `Editor`.



Dentro del editor nos aseguraremos de que se encuentre marcada la opción `Input type: Assembly`.



1.2.3. Control de la simulación

Por último, para controlar la ejecución del programa contamos con varias opciones en el menú superior.



1. Reinicia la simulación, devolviendo el estado al estado inicial antes de ejecutar ninguna instrucción.

2. En una ejecución paso a paso devuelve el estado del procesador al del ciclo de reloj anterior.

3. En una ejecución paso a paso avanza el procesador exactamente un ciclo de reloj.

4. Activa la ejecución automática del programa. Avanza automáticamente las instrucciones al periodo especificado.

5. Permite configurar el período de reloj de la ejecución automática.

6. Ejecuta instantáneamente el programa y deja el procesador en el estado final de la ejecución.

7. Permite compilar el programa en caso de realizar cambios.

2. Tipos de instrucciones y etapas de la ruta monociclo

En esta sección se presentarán los cuatro tipos de instrucciones que se estudian en teoría y las etapas de la CPU monociclo que las implementan.

2.1. Tipos de instrucciones

La Tabla 1 muestra algunos ejemplos de los tipos de instrucciones estudiados en teoría. Estos tipos se describen a continuación:

Instrucciones ALU (ejemplos: `add` y `addi`): Es el conjunto de instrucciones aritméticas y lógicas de la CPU RISC-V. La mayoría de este tipo de instrucciones se codifican con formato R o I. La instrucción `add` (formato R) suma el contenido de los registros fuente `rs1` y `rs2` y el resultado lo almacena en el registro destino `rd`. Por otro lado, la instrucción `addi` (formato I) almacena en `rd` el resultado de la suma del contenido del registro fuente `rs1` y un valor inmediato codificado en la instrucción.

Load (ejemplo: `lw`): Son instrucciones que cargan datos desde la memoria principal a los registros del procesador. La instrucción `lw` (*load word*) emplea un registro fuente `rs1` al cual le añade un valor inmediato `imm` para calcular la dirección de memoria desde donde se cargará el dato (una palabra de 32 bits). Una vez calculada la dirección, el dato se lee de memoria y se almacena en el registro destino `rd`.

Store (ejemplo: `sw`): Son las instrucciones encargadas de almacenar datos contenidos en los registros en memoria principal. Al igual que la instrucción `lw`, la instrucción `sw` (*store word*) calcula la dirección de memoria a la que acceder a partir del registro `rs1` y del valor inmediato. El dato contenido en el registro fuente `rs2` se almacena en esa dirección.

Instrucción	Operación	Formato
add rd, rs1, rs2	rd ← rs1 + rs2	R
addi rd, rs1, imm	rd ← rs1 + imm	I
lw rd, imm(rs1)	rd ← M[rs1+imm]	I
sw rs2, imm(rs1)	M[rs1+imm] ← rs2	S
beq rs1, rs2, imm	PC ← PC+(imm<<1) si rs1==rs2	B

Figura 1: Instrucciones del ensamblador RISC-V vistas en teoría.

Etapa	Instrucción: add rd,rs1,rs2 addi rd,rs1,imm	lw rd,imm(rs1)	sw rs2,imm(rs1)	beq rs1,rs2,inm	Salida para siguientes etapas
	rd ← rs1 + (rs2 o imm)	rd ← M[rs1+imm]	rs2 → M[rs1+imm]	PC←PC+(imm<<1) si rs1==rs2	
IF	IR ← M[PC]; PC ← PC				IR, PC
ID	Decodificar(IR); A ← BR[rs1]; B ← BR[rs2]; I ← Generar-Inmediato(imm)				A (rs1), B (rs2), I (imm)
EX	ALUout (result)← A+ (B/I)	ALUout (Dirección) ← A + I		ALUout (Dest.) ←PC+I; cond ← (A == B)	ALUout (result, dir o dest) y cond
ME		LMD ← M[dir]	B → M[dir]	Si cond PC ← target	LMD
WB	BR[rd] ← ALUout o LMD				

Figura 2: Etapas de las instrucciones.

Branch (ejemplo: `beq`): Las instrucciones de salto condicional o *branch* permiten alterar el flujo de ejecución de un programa. La instrucción `beq` (branch if equal) compara si el valor de dos registros fuente `rs1` y `rs2` es igual, y en caso de cumplirse dicha condición, se suma al PC (contador de programa) el contenido de un valor inmediato desplazado un bit a la izquierda. Este desplazamiento se realiza porque en RISC-V las instrucciones se sitúan en direcciones pares de memoria (desplazar un bit a la izquierda es equivalente a multiplicar por dos).

2.2. Etapas de las instrucciones

Como se explica en teoría, las instrucciones pasan por varias etapas para ejecutarse. Estas etapas se muestran en la Tabla 2 y se describen a continuación:

IF: Instruction Fetch. Durante esta etapa se accede a la dirección de memoria apuntada por el valor del contador de programa (PC) y se trae al procesador dicha instrucción (IR).

ID: Instruction Decode. En esta etapa la instrucción se decodifica. En paralelo, se leen los registros fuente (`rs1`, `rs2`), cuyos contenidos se envían por las salidas A y B, y se genera el valor inmediato (I) a partir de su codificación en la instrucción.

EX: Execution. En esta etapa se realizan las operaciones aritméticas o lógicas necesarias para la ejecución de la instrucción. El detalle de las operaciones depende del tipo de instrucción:

Instrucciones ALU: calculan el resultado de la instrucción (salida ALUout) que debe escribirse en el registro destino.

Load y Store: calculan la dirección de acceso a memoria (salida ALUout).

Branch: calculan la condición de salto (salida cond) y el destino de la instrucción de salto (salida ALUout).

ME: Memory. En esta etapa, las instrucciones Load y Store acceden a memoria para, respectivamente, cargar (salida LMD para siguiente etapa) o almacenar los datos que se pasan por B (contenido del registro rs2). Por otro lado, los saltos condicionales

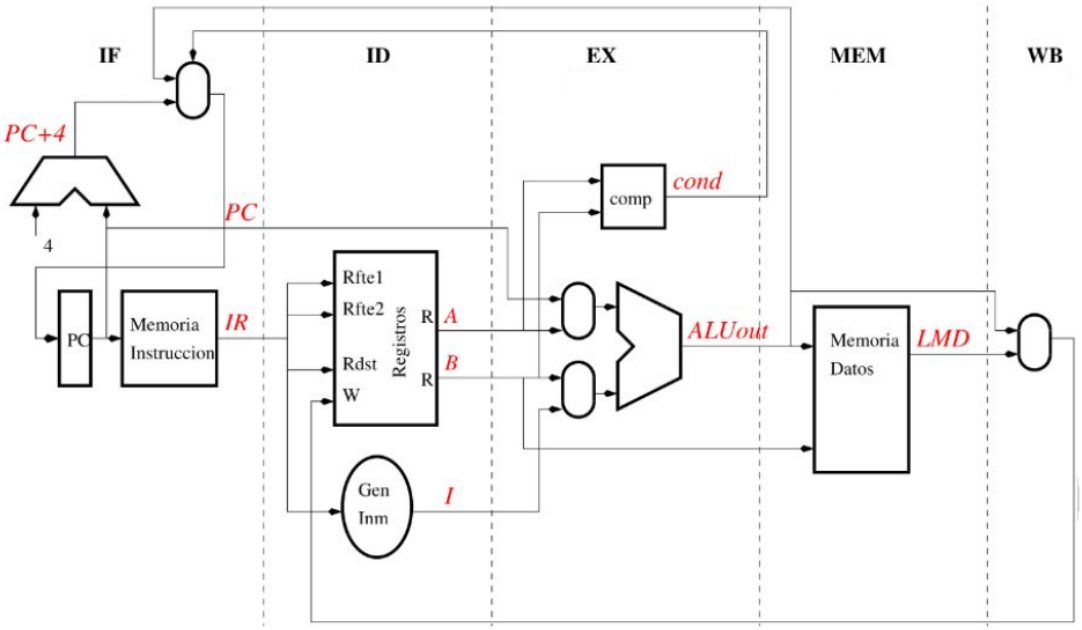


Figura 3: Etapas de la CPU básica monociclo.

modifican el PC para que apunte al destino del salto (ALUout) si la condición de salto (cond) resulta verdadera. Nótese que en esta etapa, las instrucciones ALU no realizan trabajo alguno.

WB: Writeback. Esta etapa sólo se aplica a las instrucciones que pueden almacenar datos en el banco de registros (también denominadas instruccioens productoras). En general, estas son las instrucciones ALU y las instrucciones Load. Durante la misma, se escribe en el registro destino (`rd`) el resultado de la instrucción ALU (procedente de ALUout) o el dato que las instrucción Load carga de memoria (procedente de LMD).

La Figura 3 muestra los componentes y la organización lógica de las etapas que conforman la CPU básica monociclo.

3. Programa ejemplo

Se ha preparado un programa de ejemplo (Listado 1) que busca, en unas pocas líneas, ejemplificar el funcionamiento de los cuatro tipos de instrucciones para comprender su ejecución en la CPU básica. El programa realiza los siguientes pasos:

Línea 16. La instrucción `lui x10, 0x10000` inicializa el registro (`x10`) con la dirección base del segmento de datos `0x10000000`, justo donde se encuentra la variable a.

Líneas 19 y 20. Las instrucciones `lw x11, 0(x10)` y `lw x12, 4(x10)` leen dos valores enteros desde la memoria (variables a y b) y los almacenan en los registros `x11` y `x12`, respectivamente.

Líneas 23 y 24. Las instrucciones `add x13, x11, x12` y `addi x14, x11, -1` calculan $(a + b)$ y $(a - 1)$, guardando los resultados en `x13` y `x14`, respectivamente.

Listing 1: Programa de ejemplo — `Prac01.s`

```
# ============================================================
#  Practica 1 - AIC-IA: Instrucciones basicas RISC-V
# ============================================================

    .data  # (datos comienzan en 0x10000000)
a:  .word  0x00A0FF81      # (dato a)
b:  .word  0x00000001      # (dato b)
c:  .word  0x00000000      # (resultado)
d:  .word  0x00000000      # (otro resultado)


    .text  # (código comienza en 0x00000000)
    .globl main
main:
    # --- PREPARACION ---
    lui x10,0x10000        # x10 <- 0x10000000 (apunta a a:)

    # --- LOAD: Lectura de datos desde memoria ---
    lw  x11, 0(x10)        # x11 <- MEM[a]
    lw  x12, 4(x10)        # x12 <- MEM[b] (desplazamiento de 4 bytes)

    # --- ALU: Operaciones aritmeticas ---
    add x13, x11, x12      # x13 <- x11 + x12
    addi x14, x11, -1      # x14 <- x11 - 1

    # --- STORE: Escritura del resultado en memoria ---
    sw x13, 8(x10)         # MEM[c] <- x13 (desplazamiento de 8 bytes)

    # --- BRANCH: Salto condicional ---
    beq x14, x0, iguales   # si x14 == 0, saltar a la etiqueta 'iguales'

    # Si x4 != 0
    addi x15, x0, 0        # x15 <- 0
    j fin                  # salto incondicional a 'fin'

iguales:
    addi x15, x0, 1        # x15 <- 1

fin:
    sw x15,12(x10)         # MEM[d] <- x15 (desplazamiento de 12 bytes)

    # --- FIN DE EJECUCION ---
    addi x17, x0, 10       # x17 (a7) = 10 (Codigo para syscall Exit)
    ecall                  # Llamada al sistema (detiene el simulador)
```

Línea 33. La instrucción `sw x13, 8(x10)`, guarda el resultado de la suma $(a + b)$ en memoria.

Líneas 30–37. La instrucción `beq x14, x0, iguales` salta a la instrucción `addi x15, x0, 1` si $x14 = x0$. Si no, continúa con la ejecución de `addi x15, x0, 0` y salta al final del programa. Consecuentemente, el valor del registro x15 queda como '1' si el contenido de x14 es nulo o '0' en otro caso.

Líneas 43 y 44. Se almacena el contenido de x15 en memoria y se finaliza la ejecución del programa.

4. Ejercicios

Utilizando el simulador, cargaremos el programa ejemplo. En primer lugar, observaremos cómo se codifican algunos datos e instrucciones en la memoria antes de comenzar la ejecución. Seguidamente, a nivel de programa, analizaremos el efecto de algunas instrucciones en el estado de la ejecución (registros y memoria). Finalmente, para esas mismas instrucciones, investigaremos el funcionamiento interno de la CPU monociclo que causa los cambios en el estado.

4.1. Ejercicio 1: mapa de memoria inicial

Como se explica en teoría, en memoria, los programas disponen de segmentos de datos (delimitados por la directiva `.data` del ensamblador, ver Listado 1) y de código (directiva `.text`). El segmento de datos contiene los valores de las variables del programa, mientras que el segmento de código contiene los códigos binarios (código máquina) de las instrucciones del programa.

En el simulador, el contenido de estos segmentos se puede observar siguiendo los siguientes pasos:

1. En el menú de la izquierda seleccionamos la vista `Memory`.



2. Una vez abierto el visor de la memoria, a la derecha del todo tenemos el desglose del mapa de memoria llamado `Memory map`. En este menú se encuentran tanto su rango de direcciones como su tamaño. Podemos observar 3 regiones:

Memory map

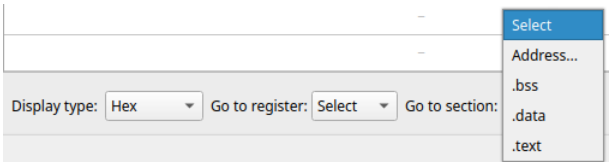
Name	Size	Range
.text	52	0x00000000 - 0x00000033
.data	12	0x10000000 - 0x1000000b
.bss	0	0x11000000 - 0x10ffffff

`.text`: Segmento de texto; contiene el código.

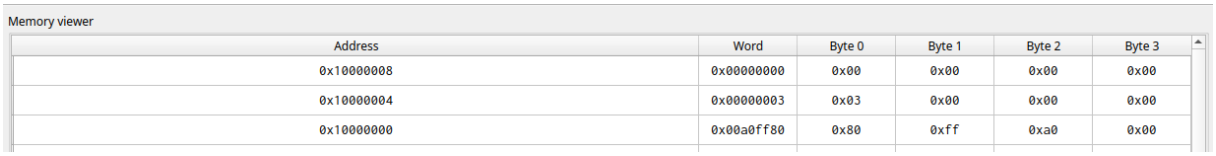
`.data`: Segmento de datos; almacena los datos inicializados definidos en este segmento.

`.bss`: Contiene las variables no inicializadas por el programador (no es importante en la práctica).

3. En la parte inferior contamos con un menú que nos permite desplazarnos a regiones específicas de la memoria para poder observar en detalle el contenido de dichas secciones.



En el apartado central de la sección de memoria llamado `Memory viewer` podemos ver en detalle las direcciones de memoria y su contenido.



Utiliza el visor de la memoria para contestar a las siguientes preguntas. **IMPORTANTE**: En vez de simplemente copiar los datos en la vista `Memory` ofrecidos por el simulador, intenta responder a las preguntas a partir del código fuente del programa ejemplo. Excepto el código máquina de la instrucción en P2, toda la información se puede deducir del código fuente.

P1. Localiza el dato `b` del segmento de datos en la memoria principal. ¿En qué dirección de memoria se encuentra almacenado y cuál es su valor en hexadecimal?

Dirección: 0x00000004 Valor: 0x00000001

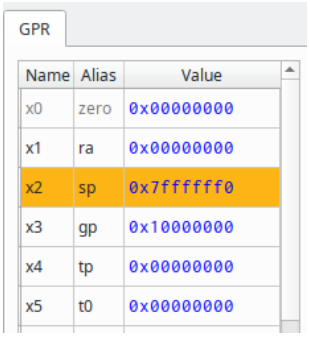
P2. Localiza la primera instrucción del programa (`lui x10, 0x10000`) del segmento de código en la memoria principal. ¿Cuál es su dirección de memoria y cuál es su código máquina?

Dirección: 0x00000000 Código máquina: 0x10000537

4.2. Ejercicio 2: efecto de la ejecución de las instrucciones en registros y memoria

El objetivo de este ejercicio es entender que efecto tienen las instrucciones en el estado de la memoria y los registros de la CPU. Para poder observarlo en el simulador, podemos usar la vista `Memory` del ejercicio anterior y la vista `Editor`. En la parte de la derecha

de esta última vista veremos una tabla llamada `GPR` que contiene el nombre, el alias y el valor almacenado por cada uno de los registros durante la ejecución.



Para analizar el efecto de cada instrucción, las ejecutaremos paso a paso usando el control de simulación explicado en la Sección 1.2.3. Utiliza la información en las vistas `Memory` y `Editor` para contestar las siguientes preguntas. **IMPORTANTE**: Intenta rellenar las tablas siguientes antes de ejecutar las instrucciones correspondientes. Luego ejecútalas y comprueba si tu razonamiento era correcto o no.

P1. Instrucciones ALU: Para las siguientes instrucciones, anota su PC, así como el identificador y valor de sus operandos fuente y registro destino. En la columna **Reg/I**, indica o bien el identificador del segundo registro fuente o un inmediato (I). Si un operando no se usa, anota *N/A*.

Instrucción	**PC**	**Fte. 1 (rs1)**		**Fte. 2 (rs2/inm)**		**Destino (rd)**	
		Reg	**Valor**	**Reg/I**	**Valor**	**Reg**	**Valor**
`add x13, x11, x12`	0x0000000c	x11	0x00a0ff81	x12	0x00000001	x13	0x00a0ff82
`addi x14, x11, -1`	0x00000010	x11	0x00a0ff81	-1	-1	x14	0x00a0ff80

P2. Load y Store: Para las siguientes instrucciones, anota su PC, así como el identificador y valor de sus operandos fuente (**rs1** y **rs2**), el valor del **inmediato** y su registro destino (**rd**). Para la memoria, indica la dirección accedida, el valor del dato y si es una operación de Lectura o Escritura (**L/E**). Si un operando no existe o no se utiliza en la instrucción (por ejemplo, `lw` no tiene segundo registro fuente **rs2**, y `sw` no tiene registro destino **rd**), anota *N/A*.

Instrucción	**PC**	**Fte. 1 (rs1)**		**Fte. 2 (rs2)**		**Inm.**	**Destino (rd)**	
		Reg	**Valor**	**Reg**	**Valor**		**Reg**	**Valor**
`lw x11, 0(x10)`	0x0...04	x10	0x1000000(	N/A	N/A	0	x11	0x1000000(
`sw x13, 8(x10)`	0x0...014	x10	0x1000000(	N/A	N/A	8	x13	???

Instrucción	**Memoria**		
	Dir	**Dato**	**L/E**
`lw x11, 0(x10)`	0x10000000	0x00a0ff81	L
`sw x13, 8(x10)`	0x10000008	0x00a0ff82	E

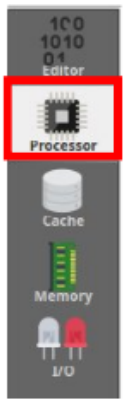
P3. Branch: Para las siguientes instrucciones, anota su PC, así como el identificador y valor de sus operandos fuente. Anota también la condición de salto **(S/N)**, el destino de la instrucción de salto y el PC de la siguiente instrucción a ejecutar.

Instrucción	**PC**	**Fte. 1 (rs1)**		**Fte. 2 (rs2)**	
		Reg	**Valor**	**Reg**	**Valor**
`beq x14, x0, iguales`	0x10000018	x14	0x00a0ff80	x0	zero

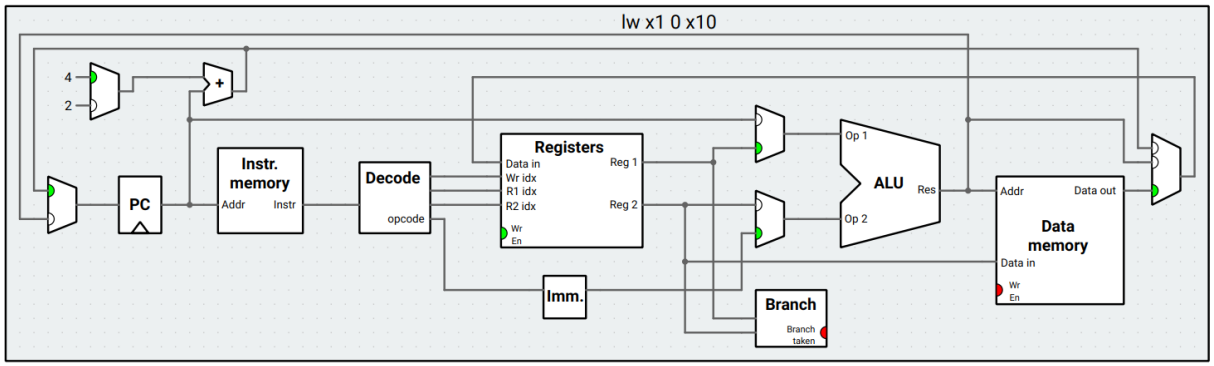
Instrucción	**¿Salta? S/N**	**Destino**	**Sig. PC**
`beq x14, x0, iguales`	N	00000024	0x1000001c

4.3. Ejercicio 3: funcionamiento interno del pipeline de la CPU

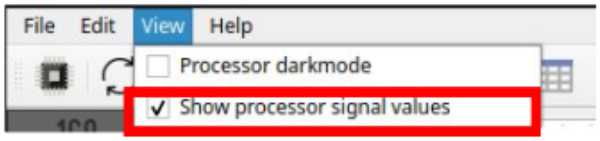
Para ejecutar las instrucciones y producir los efectos deseados en registros y memoria, el pipeline de CPU cuenta con una organización, vista en clase de teoría, de acuerdo a las etapas en la Tabla 2 y Figura 3. Esta representación es accesible en el simulador a traves de la vista `Processor`:



Dentro de esta vista, podemos observar una representación de la ruta monociclo similar a la vista en clase de teoría:



Durante la ejecución en el simulador, los valores que recorren las conexiones internas del pipeline se pueden observar marcando la opción de menú principal `Show processor signal values`:



Teniendo en cuenta el trabajo que realiza cada tipo de instrucción en las etapas (ver Tabla 2 y Figura 3), completa el valor que recorre la conexión correspondiente dentro de la ruta para la instrucción indicada. Ten en cuenta que el nombre en las columnas corresponde a los nombres que les da a las conexiones el simulador. Por ejemplo, la columna `R1 idx` corresponde a la entrada que identifica el registro fuente 1 (`Rfte1` en la ruta de la Figura 3) en el banco de registros, mientras que la columna `Res` se corresponde con la salida ALUout. **IMPORTANTE:** Como en los ejercicios previos, intenta rellenar las tablas razonando lo que hace cada instrucción en cada etapa antes de comprobar el resultado con el simulador activando `Show processor signal values`. Puedes basarte en los datos que hemos recopilado de las instrucciones en ejercicios previos.

P1. Instrucciones ALU (fuentes rs1 y rs2):

Instrucción	**Instr. memory**	**Registers**			
	Addr	**R1 idx**	**R2 idx**	**WR idx**	**Data in**
`add x13, x11, x12`					

Instrucción	**ALU**		
	Op1	**Op2**	**Res**
`add x13, x11, x12`			

P2. Instrucciones ALU (fuentes rs1 e inmediato):

Instrucción	**Instr. memory**	**Registers**			
	Addr	**R1 idx**	**WR idx**	**Data in**	**Imm.**
`addi x14, x11, -1`					

Instrucción	**ALU**		
	Op1	**Op2**	**Res**
`addi x14, x11, -1`			

P3. Load:

Instrucción	Instr. memory	Registers			
	Addr	R1 idx	WR idx	Data in	Imm.
`lw x11, 0(x10)`					

Instrucción	ALU			Data Memory
	Op1	Op2	Res	Data out
`lw x11, 0(x10)`				

P4. Store:

Instrucción	Instr. memory	Registers		
	Addr	R1 idx	R2 idx	Imm.
`sw x13, 8(x10)`				

Instrucción	ALU			Data Memory
	Op1	Op2	Res	Data in
`sw x13, 8(x10)`				

P5. Branch: Nota que el valor de `Branch taken` lo representa el simulador con el color rojo (falso) o verde (verdadero).

Instrucción	Instr. memory	Registers				Branch
	Addr	R1 idx	R2 idx	Reg 1	Reg 2	taken
`beq x14, x0, iguales`						

Instrucción	Imm.	ALU			PC
		Op1	Op2	Res	(input)
`beq x14, x0, iguales`					

Modifica el programa para que `Branch taken` sea rojo (falso) y rellena otra vez la tabla. Razona qué es lo que cambia, por qué y en qué afectará con respecto a la ejecución de la siguiente instrucción.

Instrucción	Instr. memory	Registers				Branch
	Addr	R1 idx	R2 idx	Reg 1	Reg 2	taken
`beq x14, x0, iguales`						

Instrucción	Imm.	ALU			PC
		Op1	Op2	Res	(input)
`beq x14, x0, iguales`					

Referencias

[1] David A. Patterson and John L. Hennessy. *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface*. Morgan Kaufmann, 1st edition, 2017.

[2] Morten Borup Petersen. Ripes. https://github.com/mortbopet/Ripes.

[3] Morten B Petersen. Ripes: A visual computer architecture simulator. In *2021 ACM/IEEE Workshop on Computer Architecture Education (WCAE)*, pages 1–8. IEEE, 2021.